An independent rebuild, kernel replay and statement audit of the `openai/NavierStokesAndEuler` breakdown theorems

(independently rebuilt and audited; this is *not* a claim that a Clay problem is solved)

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-26 (not peer reviewed; no person has audited any statement)

doi:10.5281/zenodo.23661381 (not peer reviewed)

AI-assistance disclosure. This preprint, the scripts, scans and Lean probe files it reports on, and the claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements for adequacy, no human referee has checked any argument, and every "independent" check below is made by an agent of the same AI pipeline as the one that wrote the paper, never by a person. The rebuild and audit were run by one agent instance; the statement-fidelity reading was written by that instance blind to an earlier fidelity audit (H9 [3]), itself by another instance; the paper stage (a further instance) re-ran no check and takes every number from the saved logs, whose hashes the ledger records. The mathematics under study was produced by an AI system (OpenAI [1]) and has, to our knowledge, not been refereed.

Abstract

OpenAI published a Lean 4 corpus, `openai/NavierStokesAndEuler` [1], whose authors present it as formalizing finite-time blowup results: for every positive viscosity, alternatives (C) and (D) of the Clay Institute's problem description [2], and a finite-time singularity for the unforced Euler equations. **We independently (of the corpus authors) rebuilt and audited this corpus. We do not claim that any Clay problem is solved, and credit for the proofs belongs to the corpus authors.** At corpus commit `f9e8bc5b`, with the corpus's own toolchain (Lean 4.34.0-rc2) and a reused, exact-revision Mathlib, we rebuilt all 2,440 corpus modules from source (609 + 1,829 solution-closure modules and 2 challenge modules; rc 0; zero warnings in the solution closures) [X 0001]. The four theorems `navier_stokes_breakdown_R3`, `navier_stokes_breakdown_periodic`, `euler_breakdown_R3` and `exists_compact_smooth_euler_singularity` depend on exactly `[propext, Classical.choice, Quot.sound]` in our own audit files [A 0001][A 0002][A 0003][A 0004]. Source and environment scans for soundness escapes found none, and each scan caught its planted negative controls [X 0002][X 0003][X 0008]. **One limitation is demonstrated, not discovered:** a planted *forged* declaration (`False` "proved" by `True.intro`, added without kernel checking) passes `#print axioms` with "does not depend on any axioms", so axiom printing alone is insufficient; a kernel replay rejects it [X 0004][C 0006]. We therefore re-ran the Lean kernel over all 88,400 corpus declarations (51 compiler-generated unsafe auxiliaries are skipped by design) (53,752 Navier–Stokes, 34,648 Euler): `REPLAY OK` in both [X 0005], and the toolchain's `leanchecker` accepts the 8 root and statement modules [X 0006] (a per-module check that is not transitive; closure coverage rests on the replay). An independent AI audit (Section 6) reproduced these results with its own scripts and found no soundness escape, statement mismatch or wrong number. The elaborated statement closures of the proved theorems (5,599 and 5,521 constants) are identical to those of the reference challenge statements up to alpha-renaming of hygienic binder names; a mutated reference is detected [X 0007]. A second statement-fidelity reading, written blind to H9, finds the Lean (C) faithful to Fefferman's (C), and the Lean (D) faithful to (D) as corrected by the Clay errata, which adds a periodic-pressure condition that makes the Lean nonexistence claim weaker than a main-text-only reading [C 0002][C 0003]; it agrees with H9 on every row [X 0010]. **Not established:** Comparator, `lean4export` and `nanoda` were not run; Mathlib is trusted, not re-checked; the diff of the challenge file against `formal-conjectures@8bf45ed` is open; the proof's mathematics was not refereed by people; the Lean kernel and toolchain are trusted; statement fidelity is a human judgement, made here by AI agents [C 0007]. Conditionally on fidelity and on the soundness of Lean and Mathlib, the formal object proves Fefferman's alternative (C); that is not a Clay determination [C 0005].

Tiers. Claims carry the Elenchus tiers: [A] a Lean statement that is kernel-checked with axioms in {`propext`, `Classical.choice`, `Quot.sound`} and that *we re-checked* (a tag attaches to the Lean statement, never to its gloss); [L] a cited source read in the cited text; [C] an argument or human-style judgement, not machine-checked; [X] an observation of a script or tool, never supporting a mathematical claim by itself. A tag such as [A 0001] refers to `ledger/ledger.json` (prefix `P2NS`). A Tier A row says what the kernel accepted and nothing about whether the statement means what a reader might think; that is Tier C (Section 3).

1 What the corpus claims, and what Clay asks

The corpus. The README of [1] says that for every positive viscosity it proves two results: on $\mathbb{R}^3$, smooth data and forcing for which no global smooth solution with uniformly bounded kinetic energy exists; on the torus $\mathbb{R}^3/\mathbb{Z}^3$, smooth periodic data and forcing for which no global smooth solution exists [L 0004]. It names these as alternatives (C) and (D) of the Clay description, and adds a smooth, compactly supported, divergence-free Euler initial velocity whose unforced solution develops a singularity in finite time. The authors' associated papers are linked from that README; *we did not read or check them.*

Fefferman's statement. In the official problem description [2], Fefferman asks for "a proof of one of the following four statements" (A), (B), (C), (D) [L 0001]. (A) and (B) assert global existence and smoothness (on $\mathbb{R}^3$ and $\mathbb{R}^3/\mathbb{Z}^3$, with $f \equiv 0$). The two *breakdown* alternatives are:

> (C) "Breakdown of Navier–Stokes solutions on $\mathbb{R}^3$. Take $\nu > 0$ and $n = 3$. Then there exist a smooth, divergence-free vector field $u^\circ(x)$ on $\mathbb{R}^3$ and a smooth $f(x,t)$ on $\mathbb{R}^3 \times [0,\infty)$, satisfying (4), (5), for which there exist no solutions (p,u) of (1), (2), (3), (6), (7) on $\mathbb{R}^3 \times [0,\infty)$."
> (D) "Breakdown of Navier–Stokes Solutions on $\mathbb{R}^3/\mathbb{Z}^3$. Take $\nu > 0$ and $n = 3$. Then there exist a smooth, divergence-free vector field $u^\circ(x)$ on $\mathbb{R}^3$ and a smooth $f(x,t)$ on $\mathbb{R}^3 \times [0,\infty)$, satisfying (8), (9), for which there exist no solutions (p,u) of (1), (2), (3), (10), (11) on $\mathbb{R}^3 \times [0,\infty)$."

Here (1) is the momentum equation $\partial_t u + (u \cdot \nabla)u = \nu\Delta u - \nabla p + f$, (2) is $\operatorname{div} u = 0$, (3) is $u(\cdot,0) = u^\circ$, (4)–(5) are Schwartz-type decay conditions on u° and f, (6) and (11) are C^∞ regularity of (p,u), (7) is bounded energy $\int |u(x,t)|^2 \,\mathrm{d}x < C$, (8)–(10) are the periodicity and time-decay conditions of the torus case. The PDF's *Errata* adds: "The further condition $p(x + e_j, t) = p(x,t)$ should be made explicit in Eqn (8)" [L 0002]. The PDF's Errata contains a second item, "Eqn (10) should read:" followed by a weak formulation; by its content it corrects the weak form (12), not the periodicity condition (10), and does not affect (C) or (D); we use (10) as velocity periodicity. Fefferman also notes that these questions are open for the Euler equations, which are not on the Clay list [L 0003]. The PDF is that fetched on 2026-10-26 (sha256 `c1b5f27b1a64705cfaf1afceea513db5deedca8a18ca56ab32e7f86445a06d0c`).

The four Lean theorems audited. In the Lean namespaces of the corpus:

- `NavierStokes.Comparator.navier_stokes_breakdown_R3` ($\nu > 0$): there are u_0, f satisfying the decay classes `InitialVelocityConditionDecay` and `ForceConditionDecay`, such that no pair (v,p) lies in the solution class `NavierStokesExistenceAndSmoothnessRn` for (ν, u_0, f) [A 0001];
- `NavierStokes.Comparator.navier_stokes_breakdown_periodic`: the same shape with the periodic data, force and solution classes [A 0002];
- `Euler.euler_breakdown_R3`: $\exists\, u_0$ with `InitialVelocityConditionDecay` such that no (v,p) lies in `EulerExistenceAndSmoothnessR3` [A 0003];
- `Euler.exists_compact_smooth_euler_singularity`: a supplementary statement in a Sobolev class on $[0,T^*)$ with a Beale–Kato–Majda-type divergence of $\int \|\omega\|_\infty$ [A 0004].

The corpus states the Navier–Stokes pair over its own copies of the definitions (`NavierStokes/ComparatorDefinitions.lean`), adapted from `formal-conjectures` commit `8bf45ed` [4], and ships a reference file `ComparatorChallenges/NavierStokes.lean` (and `Euler.lean`) with the same statements ending in `sorry`, which is what the Lean Comparator tool [5] is meant to match against the solution. We name the theorems exactly as in the corpus: the unqualified name `NavierStokes.navier_stokes_breakdown_R3` does not exist; the namespace is `NavierStokes.Comparator`.

2 Method

Each layer below addresses a different way a Lean development can fail to establish what it displays. The question for each layer is the same: what would this layer catch, and did it catch a deliberately planted defect of that kind? (Every experiment here runs a positive and a negative control.)

Layer	Catches	Does not catch
Rebuild from source (2.1)	stale or missing artifacts; code that does not compile; hidden dependence on the author's machine	a wrong statement; a flaw in Lean or Mathlib
`#print axioms` (2.2)	`sorry`, smuggled `axiom`, `native_decide` (as a surfaced auxiliary axiom)	a declaration added without kernel checking (2.4)
Source scan (2.3)	any escape *construct* in the text: `set_option debug.*`, `addDecl`, `unsafe`, `implemented_by`, ...	escapes made by means outside the pattern list
Environment scan (2.3)	flagged constants (unsafe, partial, opaque, extern, `implemented_by`, axioms) in the proof closure of the theorem	what the kernel would reject at replay
Kernel replay (2.5)	any stored declaration whose value does not have its stored type	a flaw of the kernel itself; the reused Mathlib (not replayed)
Statement identity (2.6)	a proved statement that differs from the reference statement	a reference statement that does not mean what Clay asks (that is Section 3)
Fidelity reading (3)	a mismatch with Fefferman's text, by human-style judgement	errors of that judgement; here made by AI agents

Table 1: The layers of the audit, and the failure each one is designed to catch.

2.1 Rebuild with provenance [X 0001]

Source and toolchain. Corpus: a local clone of `https://github.com/openai/NavierStokesAndEuler.git`, `HEAD` = `f9e8bc5b38b6e212696e8a30e3e91517af887bbd`, `git status -short` empty. Toolchain: `leanprover/lean4:v4.34.0-rc2`, `lean -version` reports commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`; the `lean` binary has sha256 `e8baaa71855a616dc35102 8f3ad2200051b0671f423a1696a100e809302d5550`. The build used a copy of the source (without `.git` and `.lake`) on a different disk, so nothing of the original tree is read except through the copy.

Dependencies: exact-revision reuse, trusted. All nine library packages of `lake-manifest.json` (Mathlib `85e3a25e006c35636f0e53b0e9296caca2685bc0`, plausible, LeanSearchClient, importGraph, proofwidgets, aesop, Qq, batteries, Cli) were found on the machine in a prebuilt package set whose git revisions equal the manifest exactly (script `check_revs.py`); the 7.9 GB directory was copied. `lake build -no-build Mathlib` reported "All targets up-to-date (8762 jobs)" before the build, and the build logs contain *no* "Built Mathlib" line (they show "Replayed"). **Mathlib's oleans are therefore reused, not rebuilt.** Their provenance is inferred from timestamps (`Mathlib.olean` dated three minutes after its package directory) to be Mathlib's cache artifacts; this was not otherwise verified. They are a stated trust assumption (Section 4).

The one build-only deviation. In the copy only, the `[[require]]` for `Comparator` was removed from `lakefile.toml`, and its manifest entries `Comparator` and `lean4export` from `lake-manifest.json` (script `strip_comparator_dep.py`; originals kept as `*.orig`). Reason: neither package is in the prebuilt set, and fetching them would mean downloading third-party code. Why this changes no compiled module: no corpus file imports `Comparator` or `lean4export` (a grep of all `import` lines finds only `Mathlib`, `NavierStokes.*`, `Euler.*` and the corpus's own `ComparatorChallenges`); the require only supplies the `lake exe comparator` tool. The compiler options of the corpus's lakefile were kept: Euler is built with `autoImplicit=false, warningAsError=true`, as confirmed in its `Solution.setup.json`.

The corpus's own `#print axioms` lines, emitted during the build, report `[propext, Classical.choice, Quot.sound]` for all four theorems. Artifacts: `NavierStokes/ComparatorSolution.olean` sha256 `5025fa4e63`

Target	Modules built	Result	Wall time
`NavierStokes.ComparatorSolution`	609	"Build completed successfully (9371 jobs)", rc 0	3,300 s
`Euler.Solution`	1,829	"Build completed successfully (10591 jobs)", rc 0	5,109 s
`ComparatorChallenges.NavierStokes`	1	rc 0, 2 `sorry` warnings (placeholders)	33 s
`ComparatorChallenges.Euler`	1	rc 0, 2 `sorry` warnings (placeholders)	12 s

Table 2: The rebuild (`lake build -v`, 4 cores, `LEAN_NUM_THREADS=4`). Wall times are not benchmarks: an unrelated Lean job ran on the same machine. The two solution builds contain 0 warning and 0 error lines; under Euler's `warningAsError`, any `sorry` would have failed that build. Total: 2,440 `.olean` files.

`160cea4fff8dd8f8350acabd3478cdc08563f383484fd01620de76`; `Euler/Solution.olean` sha256 `4c1131d1d7451f13977111c4116f230ac1067aa0b24e946c398cc2e71dd0102e`.

2.2 `#print axioms` on a file we wrote [A 0001]–[A 0004]

The corpus's in-file `#print axioms` lines are the authors' output. We therefore generated our own audit files (`make_dump_file.py`): each imports the built module and runs `#print axioms`, via `lake env lean`. Output, verbatim (both files rc 0):

```
'NavierStokes.Comparator.navier_stokes_breakdown_R3' depends on axioms: [propext, Classical.choice, Quot.sound]
'NavierStokes.Comparator.navier_stokes_breakdown_periodic' depends on axioms: [propext, Classical.choice, Quot.sound]
'Euler.euler_breakdown_R3' depends on axioms: [propext, Classical.choice, Quot.sound]
'Euler.exists_compact_smooth_euler_singularity' depends on axioms: [propext, Classical.choice, Quot.sound]
```

The set is exactly the whitelist. **Controls** (`EnvControl.lean`, same metaprogram) [X 0008]: a `sorry` reports `sorryAx`; a smuggled `axiom` is reported by name; `native_decide` proving a *false* equation through `implemented_by` is reported as an auxiliary axiom `..._native.native_decide.ax_1_1` (on this toolchain it is not `Lean.ofReduceBool`). All three are outside the whitelist and are caught.

2.3 Soundness-escape scans, with planted controls [X 0002][X 0003]

Source scan. We computed the transitive import closure inside the corpus: 609 modules for `NavierStokes.ComparatorSolution`, 1,829 for `Euler.Solution`, and 2,585 for both together with `NavierStokes`, `Euler` and the two challenges (615,319 lines; 309 external Mathlib/core imports). (The 2,585 scanned modules are a different number from the 2,440 built: the six roots, namely the two solutions, the umbrella modules `NavierStokes` and `Euler` and the two challenge files, have a union closure that contains corpus modules lying in neither solution closure.) A script (`source_scan.py`) strips comments and docstrings, blanks string literals, and counts code hits for 24 pattern groups: `sorry`, `admit`, `native_decide`, `ofReduceBool`/`trustCompiler`, `axiom`, `implemented_by`, `extern`, `unsafe`, `csimp`, `opaque`, `partial`, `#exit`, `set_option` of any kind (and `debug.*`, `pp.*`, `skipKernelTC` specifically), `addDecl`/`modifyEnv`/`setEnv`/`Kernel.`, `#eval`/`run_cmd`/`initialize`, `import Lean`, `command_elab`-style attributes, notation-like definitions and local instances. The 4 `sorry` hits are the intentional placeholders at `ComparatorChallenges/NavierStokes.lean:277,284` and

Pattern group	closure (2,585 modules)	all corpus files (2,659)
every escape group above except the rows below	0	0
`sorry`	4	4
`#print axioms` (the corpus's own final lines)	4	4
elab/macro/syntax group (all are `local notation` or `@[inherit_doc]`)	150 (148 + 2)	156 (154 + 2)
`attribute [local instance]`	10	11

Table 3: Code hits of the source scan (counts from `scan_closure.json` and `scan_allfiles.json`). Only the escape groups agree between the two columns.

`Euler.lean:88,184`; neither solution imports these files. The 150 (156) hits in the elab/macro/syntax group are 148 (154) `local notation` lines (such as `R3`) and 2 `@[inherit_doc]`; a line-initial grep for `elab`, `macro`, `syntax`

and `macro_rules` over the corpus finds no file. The notation of the statement files is also checked semantically by the identity test of Section 2.6. Of the 11 local instances, 4 are `Classical.propDecidable`, 6 are corpus-defined `compactInterval` instances and 1 is `CompletePartialOrder.toSupSet` in `Euler/Solution.lean`, whose elaborated type is identical to the reference (type hash 642579273). In particular there is no `maxHeartbeats`, no `debug.*` option, no `addDecl`. **Control:** one plant per category was appended to a scratch copy of `ComparatorSolution.lean` (plus an `import Lean`), and every category was caught; the unmodified copy gave no escape hit. A regex bug found during the audit (an anchored pattern whose `^\s*` spanned newlines, mis-attributing 69 notation lines) was fixed and both scans re-run.

Environment scan. A metaprogram run inside `import NavierStokes.ComparatorSolution` (resp. `Euler.Solution`) walks the full *proof* closure of the root theorems, theorem values included: 93,597 constants for Navier–Stokes (31,516 from 597 corpus modules) and 94,411 for Euler (26,213 from 1,756 corpus modules). For each reached constant it reports membership of `Lean.ofReduceBool`, `ofReduceNat`, `trustCompiler` and `sorryAx`, and whether it is `unsafe`, `partial`, an axiom, an opaque, `implemented_by` or `extern`. Result: the only axioms in either closure are `propext`, `Quot.sound`, `Classical.choice`; none of the four forbidden constants is reached; **no flagged constant comes from a corpus module**. All flags are in Lean core (58 `extern` primitives on `Nat`/`Array`/`String`, 6 `implemented_by`, 2 opaques) or Mathlib (48 resp. 51 `irreducible_def` wrappers). The corpus adds no axiom to the 15 that core Lean always declares. In the whole environment (not the proof closure) 12 (Navier–Stokes) and 39 (Euler) corpus `_unsafe_rec` constants with `partial=true` exist: compiler auxiliaries that Lean generates for the executable code of well-founded recursions; none lies in any proof closure. **Control:** a planted axiom, a `native_decide` auxiliary axiom, `implemented_by`, `sorryAx`, `unsafe def`, `partial def` and `@[extern] opaque` were all flagged.

2.4 The forgery demonstration: `#print axioms` alone is insufficient [X 0004][C 0006]

This is a demonstration of a known limitation of axiom printing, which is part of why independent checkers such as Comparator exist; we do not present it as a new vulnerability. We include it because it sets how the rest of the audit must be read: the axiom verdict of Section 2.2 means little unless something re-runs the kernel. We explain the mechanism step by step, for a reader who does not use Lean.

1. **Lean normally type checks every declaration with its kernel before adding it to the environment.** A theorem is a pair (statement, proof term), and the kernel checks that the term has the stated type. `#print axioms` is a separate, cheap query afterwards.
2. **That check can be switched off.** Metaprogramming code can call `addDecl` under `set_option debug.skipKernelTC true`, which stores a declaration the kernel never looked at. Our control (`A2Forged.lean`) adds a "theorem" `A2Forged.forged_false : False` whose proof term is `True.intro`, whose type is `True`, not `False`:

```
set_option debug.skipKernelTC true in
run_cmd liftCoreM <| addDecl (.thmDecl
  { name := `A2Forged.forged_false, levelParams := [], type := mkConst ``False,
    value := mkConst ``True.intro })
theorem A2Forged.uses_forged : False := A2Forged.forged_false
#print axioms A2Forged.uses_forged
```

3. **The compiled file stores it.** The `.olean` serializes the stored declaration like any other; a fresh file that imports the module sees `A2Forged.uses_forged : False` (`#check` succeeds, rc 0).
4. **`#print axioms` does not look.** It walks the constants referenced by a declaration and collects the axioms among them. The forged proof references only `True.intro`, an ordinary constructor, so the output is `'A2Forged.uses_forged' does not depend on any axioms`, the same line as for a fully honest proof.
5. **Only a replay catches it.** A checker that re-runs the kernel on every stored declaration compares the stored type with the stored value. Both the toolchain's `leanchecker A2Forged` and our replay script exit 1 with: `(kernel) declaration type mismatch, 'A2Forged.forged_false' has type True but it is expected to have type False`.

Consequently an axiom report of "only the three standard axioms" is evidence only together with (i) a kernel replay of the stored declarations (Section 2.5) and (ii) a source scan, which finds in the corpus no `skipKernelTC`, no `addDecl`, no `set_option` at all (Section 2.3). The corpus passes both.

2.5 Full kernel replay and `leanchecker` [X 0005][X 0006]

Replay. Our script `A2Replay.lean` uses the same mechanism as `leanchecker` (`Lean.Replay`, `Kernel.Environment.replay`) with one shared base environment. It reads every declaration stored in the `.olean`, `.olean.server` and `.olean.private` files of each corpus module in a closure, imports *only* the non-corpus dependencies (Mathlib and below), and replays all corpus declarations through the kernel (pinned to a single core, `/usr/bin/time -v`). Together 88,400 declarations. **Scope of the replay:** all

Closure	Modules	Declarations	Result	Wall	Max RSS
Navier–Stokes	609	53,752	`REPLAY OK` in 621,964 ms, rc 0	11:33	8.1 GB
Euler	1,829	34,648	`REPLAY OK` in 933,056 ms, rc 0	15:42	9.8 GB

Table 4: Kernel replay of the corpus (base environment: 774,999 constants).

of these stored declarations were sent to the kernel by `Lean.Replay`, which checks every safe declaration and skips, by design, the 51 compiler-generated `_unsafe_rec` auxiliaries (12 Navier–Stokes, 39 Euler; none in any proof closure) and the 4 identical `congr_simp` duplicates. The independent audit's own replay driver attributes 53,750 and 34,646 constants to corpus modules; the difference of 2 per closure is the `congr_simp` lemmas, which Lean attributes to the Mathlib module that first declares them (this agrees with the environment scan's 53,750 / 34,646). Two corpus declarations per closure already exist in the base environment: `HasDerivWithinAt.congr_simp`, `HasDerivAt.congr_simp` (Navier–Stokes) and `ContinuousLinearMap.compLpL.congr_simp`, `AddMonoidHom.compLeftContinuous.congr_simp` (Euler), automatically realised `congr_simp` lemmas for Mathlib constants; for these four the base and corpus type hashes are equal, so they are accepted on that equality plus the Mathlib trust assumption, not re-checked. **Controls:** the forged module is rejected (rc 1, message above); the positive control (`NavierStokes.ComparatorDefinitions`, 95 declarations) replays OK.

leanchecker. The checker shipped with the toolchain replays each named module's declarations into that module's own imported environment. We ran it under `lake env` on the 8 modules `NavierStokes.ComparatorSolution`, `ComparatorR3Theorem`, `ComparatorTheorem`, `ComparatorDefinitions`, `Euler.Solution`, `Euler.SolutionDefinitions` and the two `ComparatorChallenges` modules: all rc 0 (15–17 s each). The challenge modules pass because `sorryAx` is a legitimate kernel axiom; it is the axiom check, not the kernel, that rejects `sorry`. We did not run it on all 2,438 solution modules (about 15–25 s of environment import each, 10+ CPU-hours); the replay script covers the same per-declaration kernel check in 27 minutes. **Precisely what this is:** `leanchecker` and our replay drive *the same Lean kernel* through the same replay mechanism. They are two drivers of one kernel, not two independent kernels. **Scope:** `leanchecker` without `-fresh` checks only the named module's own declarations against its imports; a forged lemma in an imported module would pass it (auditor control: Section 6), so the 8-root run cited here says nothing about the other 2,430 or so modules and closure coverage rests on the full replay. A second kernel implementation (`nanoda`) is not installed here and was not run.

2.6 Elaborated-statement identity, with a mutation control [X 0007]

The proved theorems are stated over the solution-side copies of the definitions, not over the reference challenge file, so a textual comparison would not be enough: notations, instances and auto-bound names could make two identical-looking texts elaborate differently. We compared the *elaborated kernel terms*. For each pair of root theorems we took the transitive *statement closure*: the types of all constants, the values of definitions and opaques, and inductive/constructor links, but not theorem proofs. For every constant we dumped its kind, universe parameters and the `Expr.hash` of its type and value; for corpus constants we also dumped the full raw expression text. This was done in two separate environments: (1) the *reference*, i.e. the verbatim `ComparatorChallenges/*.lean` source with the dump appended (importing Mathlib only), and (2) the *solution*, `import NavierStokes.ComparatorSolution` resp. `Euler.Solution`. The comparison (`compare_dumps.py`) normalises only the module component of hygienic binder names (`x._@._stdin.N._hygCtx._hyg.M` against `x._@.NavierStokes.ComparatorDefinitions.N._hygCtx._hyg.M`), which is alpha-renaming. **Negative control:** in a copy of the reference, the energy bound $(\int \|v\|^2) < E$ was changed to $\leq E$; the comparison then reports exactly

Pair	Constants (ref./sol.)	Only ref.	Only sol.	Differing	Text-compared
Navier–Stokes (C)+(D)	5,599 / 5,599	0	0	**0**	23
Euler (both)	5,521 / 5,521	0	0	**0**	25

Table 5: Statement identity after alpha-normalisation. Before normalisation, 4 Navier–Stokes constructor types and 3 Euler constants (`toL2`, `toL2._proof_3`, `exists_compact_smooth_euler_singularity`) differ in text only in those hygienic names; their `Expr.hash` is equal.

one differing constant, `NavierStokesExistenceAndSmoothnessRn.mk` (type hash 4044432975 → 666613966). **Scope:** `Expr.hash` stands in for the full term on non-corpus (Mathlib/core) constants, so a hash collision there is possible in principle; the corpus constants are additionally compared as text. This is not Comparator (no `lean4export`, no `nanoda`, no sandbox): it checks the same property, that the proved statement *is* the reference statement, by a different implementation.

3 Second statement-fidelity audit

Identity with the reference statement (Section 2.6) says nothing about whether the reference statement means what Clay asks. That is a judgement. H9 [3] (2026-10-25) was a first reading; the present reading was written *before* we opened H9 (the comparison is the last section of `FIDELITY2.md`). It compares Fefferman's text [2] (saved extraction; the PDF's text includes the errata) with the challenge file, the solution definitions, and semantic probes compiled against the pinned Mathlib: under `open ContDiff`, the notation ∞ elaborates to `WithTop.some top`, i.e. C^∞ and not the analytic ω (an `example` using `decide` proves $\infty \neq \omega$); Δ on $\mathbb{R}^3 \to \mathbb{R}^3$ is the componentwise vector Laplacian $\sum_i D^2 w(x)[e_i, e_i]$ (via Mathlib's `laplacian_eq_iteratedFDeriv_orthonormalBasis`); `gradient` is the Riesz representative of `fderiv`; the norm on $\mathbb{R}^3 \times \mathbb{R}$ is the max norm (`rfl`) [X 0009]. (One `#check` line of the probe file names a nonexistent lemma and reports an unknown-identifier error; the assertions are the `example` lines, which compile.)

Caveat on (D): periodic pressure [C 0003]. Requiring the pressure to be 1-periodic restricts the admissible solutions, which makes the nonexistence claim *weaker* than nonexistence read against the main text of Clay's (D), where p is unrestricted. The weakening is substantive: without periodic pressure a pressure $p = -g(t) \cdot x$ with a spatially constant flow is a classical non-uniqueness device. The Clay errata itself asks for exactly the condition ("should be made explicit in Eqn (8)"), so the Lean statement follows the official corrected statement. The errata attaches it to (8), the data hypotheses, though p is an unknown; reading it as a condition on accepted solutions is the only coherent reading we found, and it is the Lean's.

The (C) witness: zero data, nonzero force [C 0002]. The proof's witness takes zero initial data and a nonzero (rescaled) force. Fefferman's (C) explicitly allows a smooth f satisfying (5), so this is admissible. It also means that (C) is not the negation of (A): (A) takes $f \equiv 0$. Criticism that the result is "forced, not unforced" is therefore not a fidelity defect against (C) as Clay states it (it would bear on (A)/(B)); that is our reading of the PDF, not a claim about the corpus's mathematics.

Agreement with H9 [X 0010]. After the table was written we read H9 [3]. On every row of (C) both audits mark MATCH. For (D) both find a match with the errata-corrected text and both note that periodic pressure in the solution class makes the Lean (D) weaker than a reading of the main text alone (H9: "DIFFERENT from the main text, MATCH with the errata"; H9's overall verdict FAITHFUL_WITH_CAVEATS). *No disagreement on any row.* Differences are of coverage, not of verdict: (a) H9's caveat on the Mathlib version (it read ∞ and Δ in a Mathlib v4.32.2 copy and compiled nothing) is resolved here by the probes against the pinned Mathlib; (b) H9's caveat that it did not check whether the solution proves *this exact* statement is resolved without Comparator by the identity test of Section 2.6; (c) H9 adds an informal ν-scaling remark that $\forall\nu$ and $\exists\nu$ are equivalent, which we did not use, since the Lean is the strong ($\forall\nu > 0$) reading; (d) H9 argues informally that the solution class is not empty (for instance $v = p = 0$ for zero data and force); we agree; the independent audit later mechanised it (Section 6); (e) H9 did not cover Euler; (f) neither audit diffed the challenge file against

Clay ingredient	Lean (A2 reading)	Verdict
(C) on $\mathbb{R}^3$ [C 0002]		
$\nu > 0$, $n = 3$	$\forall\, \nu > 0$, $\mathbb{R}^3$ = `EuclideanSpace Real (Fin 3)` (data may depend on ν)	match
u° smooth, div-free	`ContDiff Real infty u0`; $\nabla \cdot u_0 = 0$ (trace of `fderiv`; no junk value since u_0 is smooth)	match
(4) decay of $\partial^\alpha u^\circ$	$\forall m, K\ \exists C\ \forall x$: $\|D^m u_0(x)\| \le C/(1+\|x\|)^K$; the m-th derivative tensor bounds every $\|\alpha\| = m$ partial and conversely; K real is harmless ($K \le 0$ follows from $K = 0$)	match
f smooth, (5) decay	`ContDiffOn` on $\mathbb{R}^3 \times [0,\infty)$; joint space–time jets bounded by $C/(1+\|x\|+t)^K$ (equivalent norms)	match
(1) momentum eq., $t \ge 0$	∂_t one-sided at $t = 0$; $D_x v\,(v) = (v \cdot \nabla)v$; $\nu\,\Delta v - \nabla p + f$ with the Riesz gradient	match
(2), (3)	$\nabla \cdot v(\cdot, t) = 0$ for $t \ge 0$; $v(x, 0) = u_0(x)$	match
(6) $p, u \in C^\infty(\mathbb{R}^3 \times [0,\infty))$	`ContDiffOn Real infty` on `univ x Ici 0` for v and p	match
(7) bounded energy	`MemLp (norm (v . t)) 2` *and* $\exists E\, \forall t \ge 0 : \int \|v\|^2 < E$; the `MemLp` conjunct only removes Mathlib's junk value 0 for non-integrable Bochner integrals	match
"no solutions"	$\neg\exists\, v\, p$ over the class with exactly the fields above; nothing extra demanded of a solution	match
(D) on $\mathbb{R}^3/\mathbb{Z}^3$ [C 0003]		
data (8): periodic u°, f	`isOnePeriodic`: $u_0(x + e_j) = u_0(x)$; $f(\cdot, t)$ 1-periodic for $t \ge 0$	match
(9) time decay of f	$\|D^m f\| \le C/(1+t)^K$ on $\mathbb{R}^3 \times [0,\infty)$	match
(10), (11)	$v(\cdot, t)$ 1-periodic for $t \ge 0$; v, p smooth	match
errata: $p(x + e_j, t) = p(x, t)$	`isOnePeriodic_pressure`, a field of the *solution* class	match with errata (caveat)
Euler (not a Clay prize problem) [C 0004]		
$\nu = 0$, $f \equiv 0$ analogue of (C)	`euler_breakdown_R3`: (1) with $\nu = 0, f = 0$, (2), (3), (4), (6), (7); `SolutionDefinitions.lean` is code-identical to the challenge file	match
supplementary statement	`exists_compact_smooth_euler_singularity`: no Clay counterpart; solution class read, not audited against the literature	Lean statement only

Table 6: The second fidelity table (written before reading H9). Judgement by an AI agent, tagged Tier C. Mismatches found: none.

`formal-conjectures@8bf45ed`. Both audits are by AI agents of one pipeline, so agreement raises our confidence in the table only modestly.

4 What is not established

- **Comparator, `lean4export` and `nanoda` were not run** [C 0007]. The corpus ships a Comparator configuration; we removed its `require` from our build copy because neither package was in the prebuilt set (REBUILD) and because the Comparator build available on this machine targets Lean 4.34.1 while the corpus pins 4.34.0-rc2 (kernel-replay notes). Both reasons are documented, in different notes; we do not rank them. Our statement-identity test and kernel replay are different implementations of two of Comparator's checks, not Comparator.
- **The kernel replay and `leanchecker` are one kernel.** They would not detect a bug in the Lean 4.34.0-rc2 kernel or toolchain, which are trusted.
- **Mathlib 85e3a25e and its dependencies are trusted, not re-checked.** Their oleans were reused (provenance inferred from timestamps, Section 2.1); neither rebuilt nor replayed. Replaying Mathlib (hours, single-threaded) was not attempted.
- **The diff against `formal-conjectures@8bf45ed` is open.** The challenge file declares itself copied from that commit; we did not diff it. H9 reports that upstream has since renamed the directory (`Millenium` to `Millennium`) and links to `openai/NavierStokesAndEuler@8937a8f`; whether that revision equals the audited `f9e8bc5b` was not checked.
- **The mathematics of the proof has not been refereed by people**, by us or, to our knowledge, by anyone we could cite. We read neither of the corpus authors' papers, so nothing here compares the natural-language argument with the Lean development. H9 reports a 2026-10-06 preprint (Bastounis, Circelli, Hansen, arXiv:2610.08144) that, according to its alphaXiv text, finds intermediate Lean lemmas that differ from the natural-language paper (for example a derivative-count of $m + 5$ against $m + 4$) and

does not claim the top-level statements misstate the Clay problem; we did not read it ourselves and cite it only as relayed.

- **Statement fidelity is a human judgement.** Both audits are by AI agents; no person has certified that the Lean statements mean what Clay's text means. The gate flags each Tier A row for this (Section 8).
- `leanchecker -fresh` **was not run.** Re-checking the reused Mathlib subset (about 775k constants) from an empty environment was estimated at hours and attempted neither by the producer nor by the auditor.
- **The planted controls do not exhaust the escape space.** Each layer caught the defects planted for it; this does not show that no other kind of escape exists.

5 Implications, stated carefully

We separate three questions. (1) *Does the corpus build, and does the kernel accept it?* Yes, on the evidence of Sections 2.1–2.5: it rebuilds from source, depends on the three standard axioms, and all of its 88,400 stored declarations were sent to the kernel by `Lean.Replay`, which checks every safe declaration and skips, by design, the 51 compiler-generated `_unsafe_rec` auxiliaries (12 NS, 39 Euler; none in any proof closure) and the 4 identical `congr_simp` duplicates, with no soundness escape found by our scans. (2) *Is the proved statement the stated challenge statement?* The elaborated terms are identical up to alpha-renaming (Section 2.6). (3) *Is the challenge statement Fefferman's (C)?* Our reading, like H9's, says yes, with the (D) caveat; this is a human-style judgement [C 0002].

If the Lean statement of (C) is a faithful formalization of Fefferman's (C), **and** the Lean kernel, the toolchain and the reused Mathlib are sound, **then** the kernel-checked theorem `navier_stokes_breakdown_R3` is a formal proof of Fefferman's alternative (C) [C 0005]. We make this conditional statement and no stronger one. It is not a Clay determination: the Clay Institute's own process, with its rules on publication and waiting, decides whether anything is a solution and whether a prize is due. We have not said, and do not say, that the Navier–Stokes regularity problem is solved. The credit for the proofs belongs to the corpus authors; our contribution is an independent rebuild and audit, and the explicit list of what it did not cover. For (D), the conditional carries the periodic-pressure caveat of Section 3. Euler is outside the Clay list.

6 Independent AI audit

After the first draft of this paper, a separate AI agent instance (producer $\neq$ verifier, 2026-10-26) audited it adversarially with its own scripts, running none of ours; its report is `papers/p2_navier_stokes_audit/AUDIT.md` (verdict ACCEPT_WITH_CORRECTIONS). It belongs to the same AI pipeline, so "independent" means a different instance with its own code, not a different organisation; **no human has reviewed the audit or the paper**. It found no soundness escape, no statement mismatch and no wrong number; its two blocking wording items and six non-blocking items are applied in this version (`REVISION.md`). Its re-checks, from AUDIT.md:

- **Own replay.** A replay driver written independently of `A2Replay.lean` (all corpus constants, base environment of the non-corpus modules) returned rc 0 on both closures: 53,750 corpus constants for Navier–Stokes (12 unsafe/partial skipped by `Lean.Replay`) and 34,646 for Euler (39 skipped), base environment 774,999 constants. The forged control (`AuditForge.two_eq_three : 2 = 3` stored with value `Eq.refl 2` under `debug.skipKernelTC`) is rejected with "(kernel) declaration type mismatch" (rc 1); the honest positive control and the 95-declaration `NavierStokes.ComparatorDefinitions` control pass.
- **leanchecker is per module.** `leanchecker AuditForgeUser` (an honest module importing the forged one) returns rc 0, `leanchecker AuditForge` rc 1, `leanchecker -fresh AuditForgeUser` rc 1.
- **Own** `#print axioms`, verbatim, rc 0:

```
'NavierStokes.Comparator.navier_stokes_breakdown_R3' depends on axioms: [propext, Classical.choice, Quot.sound]
'NavierStokes.Comparator.navier_stokes_breakdown_periodic' depends on axioms: [propext, Classical.choice, Quot.sound]
'Euler.euler_breakdown_R3' depends on axioms: [propext, Classical.choice, Quot.sound]
'Euler.exists_compact_smooth_euler_singularity' depends on axioms: [propext, Classical.choice, Quot.sound]
```

- **Statement identity in the kernel.** The verbatim challenge files (sha256 `0cd193b8d5cbd0266e6e2f72e68dd5abcdcf2430ebd435dd9289737e9aa7da61` for Navier–Stokes), with only the namespace renamed, were kept next to the solution in one environment and their `sorry` bodies replaced by proofs using only the solution theorems with field-by-field conversion: rc 0, so every field type is definitionally equal. Four mutants (energy `<` `E` to $\leq$ `E`; dropping the periodic-pressure field; decay `(1+`$\|x\|$`)^K` to `(2+`$\|x\|$`)^K`; Euler

`Tstar` $\leq$ `1` to `Tstar < 1/2`) all fail with the expected error. Dumps of the reference constants with and without the corpus import differ only in hygienic binder names (0 differences after normalisation).

- **Environment scan** (own metaprogram, theorem proofs included): closure totals 93,597 and 94,411 constants, closure axioms exactly the three standard ones, none of `sorryAx`, `ofReduceBool`, `ofReduceNat`, `trustCompiler` reached, 0 flagged corpus constants in the closure; the planted controls under a corpus-looking module name are all flagged.
- **Non-vacuity, mechanised** against the verbatim reference definitions (rc 0; the reference solution classes are non-empty, with $v = p = 0$ for zero data and force):

```
'audit_nonvac_Rn' depends on axioms: [propext, Classical.choice, Quot.sound]
'audit_nonvac_periodic' depends on axioms: [propext, Classical.choice, Quot.sound]
'audit_zero_data_ok' depends on axioms: [propext, Classical.choice, Quot.sound]
```

- **Rebuild sample.** Seven modules (`NavierStokes.ComparatorDefinitions`, `ComparatorTheorem`, `ComparatorSolution`, `FiveProfileMoments`; `Euler.SolutionDefinitions`, `Euler.Solution`, `LpCylinderRectangular`) were rebuilt from source with the same dependencies and invocation, rc 0, and their oleans are bit-identical to ours. This is per-module determinism against the same imported oleans, not an independent Mathlib or dependency build.
- **Fidelity spot checks** against the elaborated reference (not the source text): smoothness is C^∞, not analytic; the energy bound is strict, under Lebesgue measure on `EuclideanSpace` $\mathbb{R}$ `(Fin 3)`; `IsOnePeriodic` is $f(x + e_i) = f(x)$; periodic pressure is a field of the solution class. No disagreement with the table of Section 3.
- **Clay PDF.** The auditor fetched it independently: sha256 `c1b5f27b1a64705cfaf1afceea513db5deedca8a18ca56ab32e7f86445a06d0c`, equal to ours; the quotations of (C), (D), the first erratum and the Euler sentence are verbatim.

Not done by the auditor: Comparator, `lean4export`, `nanoda`, `leanchecker -fresh`, a Mathlib replay, the `formal-conjectures` diff, reading the authors' papers. Both AI audits and the replay drivers use one Lean kernel.

7 Reproducibility

All inputs of this paper are saved files of `results/proposal2/2026-10-26/A2/` (committed in AutoevolveAI at `d677df8b1a243118ab2df7318a737400421cc8f2`, branch `worktree-openai-math-discovery`); only `papers/p2_navier_stokes_audit/` is uncommitted. The paper stage re-ran none of the checks; the ledger evidence blobs hash the logs and audit files they rest on.

Versions and hashes. Corpus `HEAD` `f9e8bc5b38b6e212696e8a30e3e91517af887bbd`; Lean 4.34.0-rc2, commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`, binary sha256 `e8baaa71855a616dc351028f3ad2200051b0671f423a1696a100e809302d5550`; Mathlib `85e3a25e006c35636f0e53b0e9296caca2685bc0`; Clay PDF sha256 `c1b5f27b1a64705cfaf1afceea513db5deedca8a18ca56ab32e7f86445a06d0c`; `leanchecker` sha256 prefix `ccef1bfe`; oleans as in Section 2.1. Wall span of the run 01:55–07:35 UTC on 2026-10-26 (part of it idle waiting in the session).

Commands (scripts in `results/proposal2/2026-10-26/A2/evidence/scripts/`; paths inside them point to the original working directory):

```
python3 check_revs.py                    # package revisions vs lake-manifest.json (all nine match)
python3 strip_comparator_dep.py          # build copy only: drop the Comparator require and manifest entries
taskset -c 0-3 bash run_build.sh         # lake build -v for the four targets, LEAN_NUM_THREADS=4
python3 closure.py ; python3 source_scan.py   # import closure and source scan (+ make_scan_control.py for plants)
python3 make_dump_file.py ; lake env lean SolutionAudit_NS.lean   # #print axioms and soundnessScan
python3 make_challenge_dump.py ; python3 compare_dumps.py        # statement identity (+ mutated reference)
bash run_replay.sh ns ; bash run_replay.sh euler                 # lean --run A2Replay.lean, /usr/bin/time -v
bash run_leanchecker_roots.sh ; bash forged_control.sh           # leanchecker on 8 roots; forged-declaration control
```

Resources. Navier–Stokes build 3,300 s, Euler build 5,109 s (4 cores each, under contention), replays 11:33 and 15:42 (one core, 8.1 and 9.8 GB), leanchecker about 15 s per module.

8 Claim ledger

`ledger/build_ledger.py` files 26 claims: 4 at Tier A, 4 at L, 8 at C and 10 at X, each with a content-addressed evidence blob in `ledger/evidence/`: the verbatim `#print axioms` lines of our own audit files together with the hashes of the replay and `leanchecker` logs (A), verbatim excerpts of the cited text (L), the argument and its file hashes (C), and result files with `claim_licence: false` (X). The A rows rest on nothing; the C rows rest only on A and L rows (never on an X row, so the script observations are cited in the text rather than used as premises); the forged-declaration observation is filed at X, since it concerns a planted *invalid* declaration. The Elenchus gate (`tools/ledger.py ledger.json --evidence-dir evidence`, verbatim output in `ledger/gate_output.txt`) exits with code **1**: 0 blocking findings and 4 flags, all `LEDGER_UNAUDITED_TIER_A` (one per Tier A row: no person has certified that the statements mean what their glosses say). The flags are open and the audit fields are left `null`. The L rows are verbatim lines of the saved extraction of the PDF; `citation_check.py` was not run.

References

[1] OpenAI, *Finite time blowup for Navier–Stokes and Euler equations*, Lean 4 formalizations, `https://github.com/openai/NavierStokesAndEuler` (audited commit `f9e8bc5b38b6e212696e8a30e3e91517af887bbd`); README links the authors' papers. AI-generated; to our knowledge not refereed [L 0004].

[2] C. L. Fefferman, *Existence and smoothness of the Navier–Stokes equation*, Clay Mathematics Institute problem description (with errata), `https://www.claymath.org/wp-content/uploads/2022/06/navierstokes.pdf`, sha256 `c1b5f27b...` [L 0001][L 0002][L 0003].

[3] X. Callens (AI-assisted), *H9: does the Lean statement match the Clay problem? Fefferman (C)/(D) against `navier_stokes_breakdown_R3` and `_periodic`*, AutoevolveAI `results/autoresearch/2026-10-25/H9/AUDIT.md` (2026-10-25), verdict FAITHFUL_WITH_CAVEATS; first statement audit, a text-only reading.

[4] Google DeepMind, `formal-conjectures`, `https://github.com/google-deepmind/formal-conjectures`, commit `8bf45ed70d48b2b2a501de9c00b26bfa38c573ee`, file `FormalConjectures/Millenium/NavierStokes.lean` (as named in the corpus header; not diffed here).

[5] Lean FRO / leanprover, *Comparator*, `https://github.com/leanprover/comparator` (the tool the corpus's `ComparatorChallenges` are written for; not run here).

[6] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE-28, 2021; toolchain v4.34.0-rc2 used here.

[7] The mathlib Community, *The Lean mathematical library*, CPP 2020; arXiv:1910.09336; revision `85e3a25e` used here.